Übungsblatt 4: Funktionen und Stack

Aufruf-Traces, Frameadressen und Venus-Vergleich

Musterlösung — Die Zahlen sind Modellvorhersagen; Beobachtungen der konkreten Venus-Umgebung werden getrennt eingetragen.

Arbeitsweise und Zeitplanung

Die Pflichtaufgaben setzen die Einheiten 1–3 und die Erklärungen aus Vorlesung 4 voraus. C und die Multiplikationserweiterung M sind nicht erforderlich. Aufgaben 1–6 bearbeiten Sie zunächst auf Papier. Die **60 Punkte** dienen der Selbstkontrolle, nicht einer Prüfungsbewertung. Aufgabe 7 vergleicht eigene Vorhersagen mit Kurs-Venus.

Aufgabe	Inhalt	Minuten	Punkte
1	Linkwert und Registerziel	9	10
2	Registerverantwortung	8	8
3	Framegröße und Slotadressen	8	10
4	Verschachtelter Aufruf-Trace	13	14
5	Fehler isoliert erklären	9	10
6	Caller-saved Wert erhalten	8	8
7	Leaf und Calc in Venus	35	—

Für vier Schulstunden zu je 45 Minuten: 70 Minuten Vorlesung einschließlich Rückfragen, 55 Minuten Papierarbeit, 35 Minuten Venus, zehn Minuten Quiz und zehn Minuten Auswertung. Die optionale Rekursion ist zusätzliche Reserve. Die Zeiten sind Planungsannahmen; bei mehr Unterstützungsbedarf kann Aufgabe 5 nachbesprochen werden.

Gemeinsame Annahmen

RV32I bezeichnet die RISC-V-Integerbasis mit 32-Bit-Registern. Ohne komprimierte Erweiterung C ist jeder native Befehl vier Byte lang. Adressen zählen Byte, Daten liegen Little Endian. Registerarithmetik speichert die unteren 32 Bit. `ra=x1` ist return address, Rückkehradresse; `sp=x2` ist stack pointer, Stackzeiger. `a0=x10` und `a1=x11` sind Argumentregister, `s0=x8` ein saved Register und `t0=x5` ein temporäres Register.

Die ABI, Application Binary Interface, legt die Softwarevereinbarung fest: einzelne 32-Bit-Ergebnisse in `a0`, alte `t`-/`a`-/`ra`-Werte nicht allgemein erhalten, veränderte `s`-Eintrittswerte und `sp` bei Rückkehr restaurieren. Wir verwenden 16-Byte-ausgerichtete Framegrößen und einen gültigen, ausreichend großen Anfangsstack. Lehradressen sind keine zugesicherten Venus-Adressen. Nicht genannte Werte und Paddingbyte sind unbekannt.

Aufgabe 1: Rückwege aus dem alten Zustand

10 Punkte · 9 Minuten · Papier

Jeder Fall wird unabhängig betrachtet. Notieren Sie jeweils den nächsten PC, Program Counter, und den neuen beziehungsweise unveränderten `ra`-Wert.

a) Bei `PC=0x1000` wird `jal ra, Calc` ausgeführt. Das Lehrziel Calc liegt bei `0x1100` (2 Punkte).

b) Bei `PC=0x1000` und altem `ra=0x3000` wird `jal x0, Calc` zum selben Ziel ausgeführt (2 Punkte).

c) Bei `PC=0x1124` und `ra=0x1004` wird `jalr x0, 0(ra)` ausgeführt (2 Punkte).

d) Gleicher Anfang wie c), aber Befehl `jalr ra, 0(ra)`. Welche Basis wird gelesen, bevor der Linkwert geschrieben wird? (2 Punkte).

e) Welche Zielmuster bilden `jalr x0, 0(ra)` für `ra=0x1005` und `ra=0x1006`? Ist das zweite Ergebnis im Kursmodell durch vier teilbar? Gefragt ist die Adressrechnung, keine erfolgreiche Ausführung dieses ungültigen Ziels (2 Punkte).

Musterlösung

a) PC `0x1100`, `ra=0x1004`. Ziel und Linkwert verwenden beide den alten PC.

b) PC `0x1100`, `ra` bleibt `0x3000`. `x0` verwirft den Linkwert `0x1004`; das ist die Entwicklung des Sprungs `j`.

c) PC `0x1004`, `ra` bleibt `0x1004`. Der neu berechnete Linkwert `0x1128` wird in `x0` verworfen. Das ist `ret`.

d) PC `0x1004`, `ra=0x1128`. Zuerst wird die alte Basis `0x1004` gelesen. Dass Basis- und Linkzielregister gleich sind, ändert diese Reihenfolge nicht. Der Befehl ist daher kein gewöhnliches `ret`.

e) Zielmuster `0x1004` beziehungsweise `0x1006`. `jalr` löscht Bit 0, nicht beide unteren Bits. `0x1006` ist nicht durch vier teilbar; eine gültige Rückkehradresse wird dadurch nicht allgemein hergestellt.

Aufgabe 2: Wer erhält welchen Wert?

8 Punkte · 8 Minuten · Papier

Ordnen Sie jedem Fall eine Verantwortung zu und begründen Sie, ob eine Sicherung benötigt wird. „Sichern“ muss nicht immer einen Store bedeuten; eine passende Registerwahl oder Neuberechnung kann denselben Bedarf erfüllen (je 2 Punkte).

1. Der Caller hat einen Zähler in `t0` und liest dessen alten Wert nach einem normalen Aufruf weiter.
2. Der Callee liest `s0` nur und verändert es nicht.
3. Der Callee nutzt `s1` als Arbeitsregister und schreibt neue Werte hinein.
4. Calc braucht den beim Eintritt in `ra` stehenden äußeren Rückweg nach einem inneren `jal ra, Sum` erneut.

Musterlösung

1. Caller-Verantwortung: `t0` ist nicht als erhalten zugesichert. Der alte Zähler muss erhalten werden, weil er weiter gebraucht wird. Ein Stackbackup ist eine Möglichkeit, keine allein zulässige Lösung.
2. Kein Backup allein fürs Lesen nötig. `s0` ist callee-saved, sein Eintrittswert ist ohne Änderung bereits erhalten.
3. Callee-Verantwortung: Eintrittswert von `s1` erhalten und vor der Rückkehr wiederherstellen. Die Hardware sperrt das Register nicht und führt keine automatische Sicherung aus.
4. Calc ist gegenüber Sum selbst Caller. `jal ra` überschreibt den noch benötigten alten Rückweg. Das Beispiel sichert ihn deshalb vor dem inneren Aufruf und lädt ihn vor der eigenen Rückkehr zurück. `ra` wird dadurch nicht zu einem callee-saved Register.

Caller-saved heißt nur „alter Inhalt nicht zugesichert“. Der Wert kann unverändert bleiben oder verändert werden; er wird nicht automatisch gelöscht.

Aufgabe 3: Nutzbedarf, Frame und Slots

10 Punkte · 8 Minuten · Papier

Der gültige Stack beginnt mit `sp=S=0x2000`. Sie benötigen Sicherungen für `ra`, `s0` und `s1`, je ein 32-Bit-Wort.

a) Bestimmen Sie Nutzbedarf, kleinste Framegröße nach der 16-Byte-Konvention, Padding und den Reservierungsbefehl (4 Punkte).

b) Wählen Sie für diese Aufgabe ausdrücklich Offsets `12(sp)` für `ra`, `8(sp)` für `s0` und `4(sp)` für `s1`. Berechnen Sie die drei Wortstartadressen und den gesamten Framebereich (3 Punkte).

c) Reserviert ein innerer Aufruf einen weiteren 16-Byte-Frame, wo liegen dessen `sp` und sein Slot `12(sp)`? Kollidiert er mit dem äußeren Slot zwölf? (2 Punkte).

d) Wäre in einem Einwort-Frame prinzipiell auch Slot `0(sp)` zulässig, sofern der passende Load denselben Slot verwendet? (1 Punkt).

Musterlösung

a) Nutzbedarf zwölf Byte, Frame sechzehn Byte, Padding vier Byte. `addi sp, sp, -16` stellt `sp=0x1FF0` her. Drei Sicherungen sind keine drei einzelnen 16-Byte-Frames.

b) `ra` bei `0x1FFC`, `s0` bei `0x1FF8`, `s1` bei `0x1FF4`. Frame: `0x1FF0` bis `0x1FFF`, Padding an `0x1FF0` bis `0x1FF3`. Ein Slot ist vier Byte breit; die Tabelle benennt die Startadresse.

c) Innerer `sp=0x1FE0`, Slot zwölf bei `0x1FEC`. Das ist ein anderer Bereich. Freigabe in umgekehrter Reihenfolge: `0x1FE0` → `0x1FF0` → `0x2000`.

d) Ja. Slot zwölf ist eine Layoutwahl, keine allgemeine ABI-Pflicht. Slot null ist natürlich ausgerichtet und innerhalb desselben 16-Byte-Frames. Die Sicherungs- und Wiederherstellungsadressen müssen übereinstimmen.

Aufgabe 4: Der vollständige Calc-Trace

14 Punkte · 13 Minuten · Papier

Die folgende native **Lehrtabelle** verwendet feste Adressen, keine behauptete Venus-Disassembly. Vor dem Aufruf bei `0x1000` gilt `a0=5`, `a1=10`, `s0=99`, `sp=0x2000`. Unbekannter alter `ra`-Inhalt wird überschrieben.

```
1000  jal  ra, Calc          Calc = 1100
1004  addi t0, a0, 0         Fortsetzung in main
1100  addi sp, sp, -16
1104  sw   ra, 12(sp)
1108  sw   s0, 8(sp)
110C  addi s0, a0, 0
1110  jal  ra, Sum           Sum = 1200
1114  add  a0, a0, s0
1118  lw   s0, 8(sp)
111C  lw   ra, 12(sp)
1120  addi sp, sp, 16
1124  jalr x0, 0(ra)
1200  add  a0, a0, a1
1204  jalr x0, 0(ra)
```

Alle Adressen sind Hexzahlen. a) Füllen Sie die Zustände **nach** dem genannten Kontrollpunkt aus; PC bedeutet die nächste Adresse (je 2 Punkte). b) Welche Wörter liegen nach `1108` an `0x1FF8` und `0x1FFC`? (2 Punkte).

Musterlösung

nach Kontrollpunkt	PC	a0	s0	ra	sp
1000	1100	5	99	1004	2000
1110	1200	5	5	1114	1FF0
1204	1114	15	5	1114	1FF0
1114	1118	20	5	1114	1FF0
Loads bis 111C	1120	20	99	1004	1FF0
1124	1004	20	99	1004	2000

b) `0x1FF8` enthält `0x00000063`, den Eintrittswert 99. `0x1FFC` enthält `0x00001004`, die äußere Fortsetzung. Der innere `jal` verändert nur die Registerfassung von `ra`. Die Kopie `1004` im Frame bleibt erhalten, bis der Epilog sie liest. Danach übernimmt main den Wert 20 nach `t0`.

Aufgabe 5: Vier Fehler und ein falscher Rückweg

10 Punkte · 9 Minuten · Papier

Jeder Fall ist unabhängig. Beschreiben Sie die erste falsche Wirkung und die passende Korrektur (Fälle a)–d) je 2 Punkte).

a) Prolog `addi sp, sp, -16`, aber nach korrekten Loads nur `addi sp, sp, 8`. Eintritts-`sp=0x2000`.

b) `s0` wird bei `12(sp)` gespeichert, später aber ohne weiteren Store aus `8(sp)` geladen. Dort waren zuvor unbekannte Byte.

c) Nach der Sicherung bei `12(sp)` wird erst `addi sp, sp, 16` ausgeführt, dann `lw s0, 12(sp)`. Während des Frames war `sp=0x1FF0`.

d) Eine Funktion liefert 20 in `a0`. Ihr Epilog lädt aber anschließend den vor dem Aufruf gesicherten alten Argumentwert 5 wieder nach `a0`.

e) Erklären Sie den isolierten Fehler unten. Welches Label wird wiederholt erreicht, und warum ist dafür kein Stackfehler nötig? (2 Punkte).

```
Calc_bad:
    jal ra, Sum_bad
calc_bad_return:
    ret
Sum_bad:
    ret
```

Musterlösung

a) `sp=0x1FF8`, also acht Byte unter dem Eintritt und nicht durch sechzehn teilbar. Vollständig sechzehn Byte freigeben. Ein richtiges Ergebnis oder ra heilt das nicht.

b) Falscher Slot; `s0` erhält das unbekannte dortige Wort, nicht zugesichert den Eintrittswert. Dasselbe gespeicherte Offset zwölf laden. Unbekannt heißt nicht automatisch null.

c) Nach Freigabe ist die Loadadresse `0x200C` statt `0x1FFC`. Erst laden, dann freigeben; unverändert 12(sp) benutzt sonst die falsche Basis.

d) Das gewünschte Resultat wird mit fünf überschrieben. Ein alter noch benötigter Argumentwert müsste an anderer Stelle erhalten werden, während `a0` das neue Ergebnis trägt.

e) Der innere `jal` setzt `ra` auf `calc_bad_return`. Sum kehrt dorthin korrekt zurück; der dortige `ret` springt wegen unverändertem `ra` wieder zu sich selbst. Den älteren äußeren Rückweg erhalten und vor Calc-ret wiederherstellen. Dieses minimale Fragment benutzt keinen Frame und isoliert den ra-Verlust.

Aufgabe 6: Caller-saved konkret prüfen

8 Punkte · 8 Minuten · Papier

Das Hauptprogramm benötigt seinen alten `t0=5` nach einem Helferaufruf. `VerdoppleMitTemp` liefert für `a0=15` das Ergebnis 30 in `a0` und setzt `t0` ausdrücklich auf 999. Erlaubt ist das, weil `t0` caller-saved ist.

```
    addi sp, sp, -16
    sw   t0, 12(sp)
    li   a0, 15
    jal  ra, VerdoppleMitTemp
    lw   t0, 12(sp)
    addi sp, sp, 16
    add  a0, a0, t0
```

a) Welche Endwerte haben `a0`, `t0` und `sp` bei gültigem Eintrittsstack S? (2 Punkte).

b) Was würde die letzte Addition ohne t0-Sicherung und Wiederherstellung berechnen? (2 Punkte).

c) Warum darf nicht einfach auch das alte `a0=15` nach dem Aufruf zurückgeladen werden? (2 Punkte).

d) Würde eine normale zurückkehrende Funktion den Zähler stattdessen in `s0` halten, welche zusätzliche Erhaltungspflicht hätte sie gegenüber ihrem eigenen Caller? (2 Punkte).

Musterlösung

a) `a0=35`, `t0=5`, `sp=S`. Der Load restauriert den alten Zähler; das neue Funktionsergebnis 30 bleibt erhalten und wird mit fünf addiert.

b) Der Helfer hinterlässt 999 in `t0`; ohne Schutz ergibt sich $30 + 999 = 1029$. Ein unveränderter Wert wäre nicht zugesichert.

c) Dann wäre das Ergebnis 30 wieder durch das Argument 15 ersetzt. Die letzte Addition ergäbe 20. Alte Werte werden nur gemäß ihrem tatsächlichen Bedarf erhalten; Resultate dürfen nicht durch blindes Restore vernichtet werden.

d) Die Funktion verändert ein callee-saved Register und muss den eigenen s0-Eintrittswert vor der Rückkehr wiederherstellen. Verschieben in ein s-Register verschiebt die Verantwortung, beseitigt sie nicht.

Aufgabe 7: Leaf und Calc in Venus

35 Minuten · Rechner und Papierprotokoll

1. Schreiben Sie ein vollständiges Hauptprogramm und `Verdopple` für Eingabe 15. Der Leaf nutzt `slli a0, a0, 1` und `ret`. Vor dem Druckdienst prüfen Sie Ergebnis 30. Ausgabe: `mv a1, a0`, `li a0, 1`, `ecall`. Beenden: `li a0, 10` und `ecall`. Der Exit verhindert das Weiterlaufen in den Funktionscode.
2. Implementieren Sie Calc mit zwei Argumenten: Erst Sum aufrufen, dann das alte erste Argument addieren. Nutzen Sie den in Vorlesung und Papieraufgabe erklärten 16-Byte-Frame mit ra und s0. main setzt s0 als Erhaltungssignal auf 99.
3. Vor dem Lauf prüfen Sie den von der Umgebung gelieferten Stackzeiger und seine Ausrichtung. Erfinden Sie keinen festen gültigen Venus-Stack bei `0x2000`; das ist nur die Lehradresse der Papieraufgaben.
4. Am Label `main_after_calc` erwarten Sie das Ergebnis in `a0`, `s0=99` und den ursprünglichen `sp`. Lesen Sie die tatsächlichen Fortsetzungsadressen aus der nativen Liste. Kopieren Sie das Ergebnis vor einem Exit nach `s1` oder für die Ausgabe nach `a1`.
5. Prüfen Sie Argumentpaare $(5, 10)$, $(0, 0)$, $(-3, 8)$ und $(7, -10)$ nach erneutem Assemblieren und Neustart. Vergleichen Sie Zahlenresultat und Erhaltung getrennt.

Musterlösung

Vollständige Quellen: Leaf und Calc. Der Leaf benötigt keinen Frame. Calc sichert ra und s0, kopiert a0 nach s0, ruft Sum, addiert s0 zum Resultat, lädt beide Slots und gibt den Frame frei.

Die vier Resultate lauten 20, 0, 2 und 4. Bei allen korrekten Rückkehren gelten `s0=99` und `sp=S`. Sum liefert vorher 15, 0, 5 beziehungsweise −3. Die vollständige Calc-Datei kopiert das Ergebnis nach s1 und beendet ohne Konsolenausgabe; eine eigene Druckvariante verwendet vor dem Dienst `a1`.

Das Protokoll mit tatsächlichen Adressen und Anzeigezuständen ist selbst einzutragen. Die angegebenen Werte sind Modellvorhersagen, kein vorweggenommener Venus-Lauf.

Optionale Vertiefung: Rekursion ohne Multiplikation

Zusätzliche Reserve · nicht in den 90 Übungsminuten enthalten

Nutzen Sie mct1_vl4_rekursive_summe.s. SumDown(n) addiert eins bis n, vorausgesetzt $0 \leq n \leq 6$. Der Basisfall null ist framefrei; jede andere Aktivierung sichert ra und s0 in einem eigenen 16-Byte-Frame.

a) Sagen Sie Ergebnis und größte zusätzliche Stackreserve für $n = 0$, $n = 1$ und $n = 3$ voraus.

b) Vergleichen Sie die drei gleichzeitig aktiven nichtleeren Frames bei $n = 3$. Warum können alle Offset zwölf verwenden, ohne dieselbe Sicherung zu überschreiben?

c) Prüfen Sie am äußeren Rückkehrpunkt `s0=99` und `sp=S`. Beobachten Sie zunächst einzelne Kontrollpunkte; eine berechnete Stackreserve ist keine Hardwarezeitmessung.

Musterlösung

Ergebnisse: null, eins und sechs. Zusatzreserve: null, sechzehn und 48 Byte. Für drei entstehen Rahmen für n gleich drei, zwei und eins; der Basisfall null reserviert nichts. Die Rückgaben laufen als 0, 1, 3 und 6 zurück.

Die jeweiligen Stackbasen lauten $S - 16$, $S - 32$, $S - 48$. Slot zwölf hat damit die Adressen $S - 4$, $S - 20$, $S - 36$: gleiche Offsets, getrennte Orte. Jede Aktivierung restauriert ihren s0-Eintrittswert und äußeren Rückweg. Nach dem letzten ret stehen s0 und sp wieder wie am Eintritt, während a0 das Ergebnis trägt.